GATE CSE

OPERATING SYSTEMS

Complete Revision Notes

Concept summaries • Formulas • Comparison tables • Solved examples • GATE traps • Practice questions

Designed for	GATE Computer Science & Information Technology (CS)
Purpose	Long-term revision and exam-oriented problem solving
Coverage	Processes, scheduling, synchronization, deadlocks, memory, files and I/O

How to use these notes: First read the summary and comparison tables. Then work through each solved example with pen and paper. Finally, attempt the practice questions without looking at the answers.

Prepared as a study aid. Always cross-check the official syllabus and solve authentic GATE previous-year papers.

How to use this revision guide

- **Pass 1 — Understand:** Read chapter summaries and definitions.
- **Pass 2 — Apply:** Rework the examples without viewing the solution.
- **Pass 3 — Recall:** Cover the answers in the quick-revision tables and test yourself.
- **Pass 4 — Exam practice:** Solve official GATE PYQs topic-wise, then mixed tests.

Contents

Chapter	Coverage
1	OS fundamentals, kernel, system calls, interrupts
2	Processes, threads, IPC and context switching
3	CPU scheduling and performance metrics
4	Concurrency, critical sections, mutexes and semaphores
5	Deadlocks and Banker’s algorithm
6	Memory management, paging and segmentation
7	Virtual memory and page replacement
8	File systems and file allocation
9	I/O systems and disk scheduling
10	Formula sheet, common traps and practice questions

Scope note: The chapter checklist is a practical study outline aligned with the standard GATE CS Operating Systems syllabus areas. Detailed subtopics may be broader than the exact wording in the official syllabus. Verify the syllabus for your target year.

1. OS Fundamentals

1.1 What is an Operating System?

An Operating System (OS) is system software that manages hardware resources and provides services to application programs. It acts as an intermediary between users/applications and hardware.

- **Resource manager:** allocates CPU, memory, storage and I/O devices.
- **Control program:** supervises execution and helps prevent errors or misuse.
- **Abstraction:** gives convenient abstractions such as files, processes and virtual memory.

1.2 Kernel and user mode

Term	Revision point
Kernel	Privileged core of the OS; manages CPU, memory, devices and core services.
User mode	Restricted execution mode for applications; privileged operations are not directly allowed.
Kernel mode	Privileged mode in which the OS can execute protected instructions.
Mode bit	Hardware-supported indication of current privilege level (implementation-dependent details vary).
System call	Controlled interface through which a user program requests an OS service.

1.3 System calls

- **Process control:** create/terminate process, wait, execute program.
- **File management:** open, read, write, close, delete.
- **Device management:** request/release device, read/write device.
- **Information maintenance:** get time, process or system information.
- **Communication:** pipes, shared memory, message passing, sockets (availability depends on OS).

1.4 Interrupt, exception and system call

Event	Typical source	Key distinction
Interrupt	External/asynchronous event, e.g. timer or device	Can occur independently of the current instruction stream.
Exception / trap	Synchronous event associated with instruction execution	Examples include divide-by-zero, invalid access or deliberate trap.
System call	Program requests an OS service	Usually enters the kernel through a controlled trap/syscall mechanism.

GATE trap: A system call is not the same thing as a normal function call. A system call crosses the user/kernel protection boundary; a normal function call need not.

Quick revision

- The OS manages resources; the kernel is the privileged core.
- Privileged instructions are restricted to kernel mode.
- A timer interrupt helps the OS regain control from a running process.
- Exact syscall names and mechanisms vary by operating system and architecture.

2. Processes, Threads and IPC

2.1 Process basics

A process is a program in execution, together with its execution state and allocated resources. A program is passive code; a process is an active execution instance.

2.2 Process states

State	Meaning
New	Process is being created.
Ready	Can run, waiting for CPU allocation.
Running	Currently executing on a CPU.
Blocked / Waiting	Waiting for an event, such as I/O completion.
Terminated	Execution has finished.

```
Typical transitions: New → Ready → Running → Terminated; Running → Ready (preemption);
Running → Blocked (wait for event); Blocked → Ready (event completes).
```

2.3 Process Control Block (PCB)

- Process ID and process state.
- Program counter and CPU registers.
- Scheduling information (priority, queue links, etc.).
- Memory-management information.
- Accounting and I/O status information.

2.4 Context switch

The OS saves the state of one execution context and restores another. Context switching is overhead: during the switch, the CPU is not doing useful work for the switched user processes, although OS work is occurring.

2.5 Threads

Aspect	Processes	Threads in one process
Address space	Usually separate virtual address spaces	Share process address space
Resources	Own process-level resources	Share many process resources
Execution state	Own registers/PC and stack	Each thread has own registers/PC and stack
Communication	Often needs IPC	Shared memory is naturally available, but synchronization is needed
Failure isolation	Usually stronger isolation	A faulty thread can affect the whole process

2.6 IPC mechanisms

- **Shared memory:** processes access a shared region; fast for large data, but synchronization is required.
- **Message passing:** processes exchange messages using OS-supported send/receive mechanisms.
- **Pipe:** byte stream commonly used between related processes.
- **Socket:** communication endpoint that can work locally or across a network.

GATE traps: Threads share code/data/heap of their process but have separate stacks and register state. A context switch between threads of the same process may be cheaper, but this is not guaranteed by every OS or architecture.

3. CPU Scheduling

3.1 Essential metrics

```
Turnaround Time (TAT) = Completion Time (CT) – Arrival Time (AT)

Waiting Time (WT) = Turnaround Time – CPU Burst Time

Response Time (RT) = First CPU Start Time – Arrival Time
```

- **Throughput:** number of completed processes per unit time.
- **CPU utilization:** fraction of time CPU is busy.
- For a single CPU-bound burst with no I/O, WT = TAT – burst time. For multiple CPU/I/O bursts, use total CPU time in the relevant definition.

3.2 Algorithms

Algorithm	Rule	Strength / limitation
FCFS	Run in arrival order; non-preemptive	Simple; can cause convoy effect.
SJF	Choose shortest next CPU burst; usually non-preemptive	Minimizes average waiting time when burst lengths are known and assumptions hold.
SRTF	Preemptive SJF; smallest remaining time runs	Can improve average waiting; may starve long jobs.
Priority	Choose highest-priority ready process	Starvation possible; aging can help.
Round Robin	Each ready process gets a time quantum q	Fair response for time-sharing; very small q increases overhead.

3.3 Solved example: FCFS

Processes arrive in the given order and have these CPU bursts. All arrival times are 0.

Process	Burst
P1	5
P2	3
P3	2

```
Gantt chart: | P1 (0–5) | P2 (5–8) | P3 (8–10) |
```

Process	CT	TAT = CT – AT	WT = TAT – BT
P1	5	5	0
P2	8	8	5
P3	10	10	8
Average		7.67	4.33

3.4 Round Robin reminders

- If q is large compared with bursts, RR approaches FCFS.
- If q is extremely small, context-switch overhead may be high.
- When a process uses its whole quantum and remains unfinished, it usually returns to the ready queue.
- Always follow the problem's exact rules for arrivals at a quantum boundary and tie-breaking.

Numerical checklist: Write arrival times and bursts → construct Gantt chart → calculate CT → calculate TAT → calculate WT and RT → take averages only at the end.

4. Concurrency and Synchronization

4.1 Race condition

A race condition occurs when the result depends on the timing/interleaving of concurrent operations on shared state. A critical section is code that accesses shared data and must satisfy the synchronization requirements.

4.2 Critical-section requirements

Requirement	Meaning
Mutual exclusion	At most one process/thread is in the protected critical section at a time.
Progress	If the critical section is free and contenders exist, choosing the next entrant cannot be postponed indefinitely by non-contenders.
Bounded waiting	A process requesting entry has a bound on how many times others can enter first.

4.3 Mutex vs semaphore

Feature	Mutex	Semaphore
Purpose	Mutual exclusion / ownership lock	Signaling and/or resource counting
Value	Locked/unlocked concept	Integer count (binary semaphore has restricted values by convention)
Ownership	Typically owner unlocks	A signal may be issued by a different thread, depending on API/usage
Typical use	Protect shared data	Resource count, producer-consumer coordination

4.4 Semaphore operations

```
wait(S): atomically decrement/acquire; if unavailable, block or wait according to semantics.
signal(S): atomically release/increment and possibly wake a waiting process.
```

Exact pseudocode and whether the semaphore blocks, spins, or wakes a waiter depend on the implementation. For exam questions, follow the specified definition.

4.5 Producer–consumer pattern

```
empty = N; full = 0; mutex = 1
```

- Producer: wait(empty) → wait(mutex) → insert item → signal(mutex) → signal(full).
- Consumer: wait(full) → wait(mutex) → remove item → signal(mutex) → signal(empty).
- The order matters: reserving a slot/item before entering the critical section avoids incorrect counts and deadlock in the standard solution.

4.6 Classical problems

- **Readers–writers:** many readers may read together; writing generally requires exclusive access. Variants differ in fairness policy.
- **Dining philosophers:** competing processes need multiple resources; careless acquisition can deadlock or starve.
- **Spinlock:** a thread repeatedly checks a lock; useful only in suitable short-wait contexts, otherwise wastes CPU.

GATE traps: Atomicity of wait/signal is essential. Mutual exclusion alone does not guarantee progress or bounded waiting. A binary semaphore and a mutex can look similar but need not have identical ownership semantics.

5. Deadlocks

5.1 Four necessary conditions

- **Mutual exclusion:** at least one resource is non-shareable.
- **Hold and wait:** a process holds resources while requesting more.
- **No preemption:** resources cannot be forcibly taken away under the model.
- **Circular wait:** a circular chain of processes waits for resources held by the next process.

All four Coffman conditions must hold simultaneously for a deadlock to be possible in the classical model. Breaking any one condition prevents that form of deadlock.

5.2 Prevention, avoidance, detection

Strategy	Core idea
Prevention	Design the system so at least one necessary condition cannot hold.
Avoidance	Grant a request only if the resulting state remains safe; requires suitable information such as maximum claims.
Detection	Allow deadlocks to occur and run an algorithm to identify them.
Recovery	Terminate processes, preempt resources where possible, or roll back.

5.3 Safe state

A state is safe if there exists at least one order (safe sequence) in which every process can obtain its remaining needs, finish, and release resources. An unsafe state is not necessarily already deadlocked.

5.4 Banker’s algorithm

```
Need[i][j] = Max[i][j] – Allocation[i][j]
```

- Available: currently free instances of each resource type.
- Allocation: instances currently assigned to a process.
- Max: maximum claim of a process.
- Need: additional instances still required to finish.

Safety test: Work := Available; Finish[i] := false. Find an unfinished process i with Need[i] ≤ Work. Set Work := Work + Allocation[i], Finish[i] := true, and append i to the safe sequence. Repeat. If every process can finish, the state is safe; otherwise, no safe sequence was found.

5.5 Resource-allocation graph

- Process nodes are circles; resource-type nodes are boxes (instances may be shown as dots).
- Request edge: process → resource. Assignment edge: resource → process.
- With one instance per resource type, a cycle implies deadlock. With multiple instances, a cycle is necessary but not sufficient in general.

GATE trap: Safe ≠ deadlock-free forever. A currently safe state can become unsafe after future allocations. Unsafe ≠ deadlocked right now.

6. Memory Management

6.1 Address basics

Term	Meaning
Logical / virtual address	Address generated by the CPU for a process.
Physical address	Actual address in main memory.
MMU	Hardware that translates addresses and enforces memory protection.
Address binding	Mapping program addresses to memory addresses; may occur at compile, load or execution time.

6.2 Contiguous allocation and fragmentation

- **Internal fragmentation:** allocated block contains unused space inside it.
- **External fragmentation:** enough total free memory may exist, but it is split into separated holes.
- **Compaction:** moves processes/blocks to combine holes; requires relocation support and has cost.
- **First fit:** first suitable hole. **Best fit:** smallest suitable hole. **Worst fit:** largest suitable hole. Outcomes depend on the sequence of requests and holes.

6.3 Paging

Virtual memory is divided into fixed-size pages; physical memory is divided into same-size frames. A page table maps virtual page numbers to physical frame numbers.

```
For page size S bytes: page number = floor(virtual address / S)

Offset = virtual address mod S

Physical address = frame number × S + offset

If virtual address has m bits and page offset has n bits: page size = 2^n bytes; virtual page number uses m − n bits.
```

6.4 Worked address example

Virtual address = 13,000 bytes; page size = 1,024 bytes; page 12 maps to frame 7.

```
Page number = floor(13000 / 1024) = 12

Offset = 13000 − (12 × 1024) = 712

Physical address = (7 × 1024) + 712 = 7880 bytes
```

6.5 Page-table size

```
Number of virtual pages = Virtual address space size / Page size

Page-table size ≈ Number of virtual pages × bytes per page-table entry
```

This assumes a simple single-level page table with one entry per virtual page. Multilevel, inverted, or sparse page tables can have different storage behavior.

6.6 Segmentation

Segmentation divides a program into logical variable-sized units (e.g., code, data, stack). A segment table commonly stores a base and a limit for each segment.

```
Valid offset: 0 ≤ offset < limit; physical address = base + offset
```

Paging	Segmentation
Fixed-size pages	Variable-size logical segments
May have internal fragmentation in the last allocated page	May have external fragmentation

Paging	Segmentation
Transparent fixed-size memory units	Reflects logical program units and supports per-segment protection/sharing

6.7 TLB

A Translation Lookaside Buffer (TLB) caches recent page-table translations. On a TLB hit, translation is found in the cache; on a miss, the system must consult the page table (and possibly handle a page fault).

GATE trap: A TLB miss is not the same as a page fault. A TLB miss may find a valid page-table entry; a page fault means the required page is not currently available in the needed memory state or access is invalid, depending on the fault type.

7. Virtual Memory and Page Replacement

7.1 Demand paging

- Pages are loaded when needed rather than all at program start.
- A page fault traps to the OS; the OS checks validity/permissions, obtains the page, updates tables and restarts the instruction if the access is valid.
- If no free frame exists, a replacement policy may choose a victim page.
- Dirty pages may need to be written back before reuse.

7.2 Effective Access Time (EAT)

```
Simple model: EAT = (1 – p) × memory_access_time + p × page_fault_service_time
```

Here p is the page-fault probability. This simplified expression treats page-fault service time as the complete fault cost and assumes one normal access cost in the non-fault path. More detailed models may include restart accesses and additional memory references.

7.3 Replacement algorithms

Policy	Rule	Key point
FIFO	Replace page loaded earliest	Can exhibit Belady’s anomaly.
Optimal (OPT)	Replace page whose next use is farthest in the future	Theoretical lower bound on faults; future knowledge makes it impractical.
LRU	Replace page not used for the longest time in the past	Uses recent history; stack algorithm, so no Belady’s anomaly.
Second chance / Clock	FIFO-like queue with reference bit	Approximation to LRU; details depend on implementation.

7.4 How to solve page-fault numericals

- Write the reference string and number of frames.
- Process one reference at a time, left to right.
- If page is already present, mark a hit and update recency/reference information as required.
- If absent, count a fault; use the exact policy to select a victim when frames are full.
- Keep the frame table visible and count total faults at the end.

7.5 Belady’s anomaly

For some reference strings, FIFO can produce more page faults with more frames than with fewer frames. This is Belady’s anomaly. Stack algorithms such as LRU and OPT do not exhibit this anomaly.

7.6 Thrashing and working set

- **Thrashing:** the system spends excessive time paging rather than executing useful work.
- **Working set:** pages actively used in a recent time window; helps reason about a process’s current locality.
- Increasing multiprogramming can worsen thrashing if processes collectively need more frames than available.

GATE trap: In replacement questions, do not confuse load time with last-use time. FIFO uses load order; LRU uses most recent past use; OPT uses farthest future use.

8. File Systems

8.1 File concepts

- File metadata may include name, type, size, permissions, timestamps and storage pointers.
- Common operations: create, open, read, write, seek, close, rename and delete.
- Directory structures map names to file metadata or file-system objects.
- Access methods include sequential and direct/random access; indexed access is supported by some systems.

8.2 File allocation

Method	Advantages	Limitations
Contiguous	Fast sequential and direct access; simple mapping	External fragmentation; file growth can be difficult
Linked	Easy growth; avoids external fragmentation of file blocks	Poor random access; pointer overhead and pointer reliability concerns
Indexed	Supports direct access via index block(s)	Index storage overhead; large files may need multilevel indexing

8.3 Free-space management

- **Bitmap/bit vector:** one bit (or defined unit) tracks each allocation unit.
- **Free list:** links free blocks together.
- **Grouping/counting:** store collections or runs of free blocks efficiently.

8.4 Inodes and links

In Unix-like file systems, an inode stores metadata and references to file data, while a directory maps a filename to an inode. A hard link is another directory entry for the same inode; a symbolic link stores a path-like reference. Exact behavior depends on the file system and OS.

GATE trap: A filename is not necessarily stored inside the inode. Do not assume that deleting one name always deletes the data immediately; links and open references affect when storage can be reclaimed.

9. I/O Systems and Disk Scheduling

9.1 I/O concepts

Technique	Revision point
Programmed I/O / polling	CPU repeatedly checks device status or performs data movement.
Interrupt-driven I/O	Device signals the CPU when service is needed; reduces continuous polling.
DMA	Direct Memory Access controller transfers blocks between device and memory with limited CPU involvement.
Buffering	Temporarily stores data to handle speed or size differences.
Caching	Keeps copies of data to speed future access.
Spooling	Queues jobs/data for a device that serves requests sequentially, e.g. print jobs.

9.2 Disk access time

```
Approximate disk access time = seek time + rotational latency + transfer time (+ controller/queue overhead if specified)
```

For a disk rotating at R revolutions per minute, one rotation takes 60/R seconds. Under a uniform-arrival assumption, average rotational latency is half a rotation: 30/R seconds.

9.3 Disk scheduling algorithms

Algorithm	Rule	Caution
FCFS	Serve requests in arrival order	Fair/simple, but may travel far.
SSTF	Serve request closest to current head position	Can starve far-away requests.
SCAN	Move in one direction serving requests, then reverse	Elevator-style; boundary handling matters.
C-SCAN	Serve in one direction; return to the start without servicing on return	More uniform wait in common models; count wraparound movement only as directed.
LOOK	Like SCAN but reverse at last request in that direction	Does not travel to physical disk end unnecessarily.
C-LOOK	Like C-SCAN but jumps between extreme pending requests	Follow the question's convention for counting the circular jump.

9.4 Disk-head movement example

Queue: 82, 170, 43, 140, 24, 16, 190; initial head = 50. Under FCFS, total movement is:

```
|82–50| + |170–82| + |43–170| + |140–43| + |24–140| + |16–24| + |190–16| = 642 cylinders
```

Numerical trap: SCAN/LOOK questions must state (or imply) initial direction and whether the head travels to the disk boundary. For C-SCAN/C-LOOK, conventions for counting the wraparound jump may vary; state your assumption.

10. Last-minute Formula Sheet

10.1 Scheduling

```
TAT = CT − AT

WT = TAT − total CPU burst time

RT = first start time − AT
```

10.2 Paging and segmentation

```
Page number = floor(VA / page size)

Offset = VA mod page size

PA = frame number × page size + offset

Segmented PA = segment base + offset, valid only when offset < segment limit
```

10.3 Memory and storage

```
Number of pages = virtual address space / page size

Simple page-table size = number of pages × PTE size

Disk access ≈ seek + rotational latency + transfer

Average rotational latency (uniform arrival) = 30 / RPM seconds
```

10.4 Deadlock

```
Need = Max − Allocation
```

Safety test: repeatedly find an unfinished process with Need ≤ Work; after it finishes, Work := Work + Allocation. If all processes finish, a safe sequence exists.

10.5 Quick comparison table

Concept pair	Difference to remember
Program vs process	Passive code vs executing instance.
Process vs thread	Separate process resources/address space vs shared process resources with separate thread execution state.
Preemptive vs non-preemptive	OS may take CPU away vs process keeps CPU until it blocks/finishes/yields under the model.
Deadlock vs starvation	Circular resource waiting vs indefinite postponement due to policy/contention.
Safe vs unsafe	A safe completion sequence exists vs none is guaranteed by the state; unsafe does not mean already deadlocked.
TLB miss vs page fault	Translation cache miss vs missing/invalid page condition requiring fault handling.
Internal vs external fragmentation	Waste inside allocated unit vs free space split into holes.
FIFO vs LRU vs OPT	Oldest loaded vs least recently used in past vs farthest use in future.
SCAN vs LOOK	Go to disk boundary before reversing vs reverse at last pending request.

11. GATE-style Practice Set

Try each question before checking the answer key. These are original practice questions written in GATE style, not verbatim official previous-year questions.

Questions

Q1. A process arrives at time 2, first gets CPU at time 5, and completes at time 14. Its response time and turnaround time are:

Q2. Which scheduling policy can show Belady's anomaly?

Q3. Page size is 4 KiB. How many offset bits are needed?

Q4. Need = [2, 1], Available/Work = [3, 2]. Can a process with this Need finish?

Q5. Is a cycle in a resource-allocation graph always enough to prove deadlock?

Q6. A TLB miss always causes a page fault. True or false?

Q7. For RR, what is the likely effect of a very small time quantum?

Q8. Name the four necessary deadlock conditions.

Q9. A disk rotates at 6000 RPM. Find average rotational latency under uniform arrival.

Q10. Which file allocation method naturally supports direct access through an index structure?

Q11. In paging, VA = 5000 bytes and page size = 4096 bytes. Find page number and offset.

Q12. Does a safe state mean the system can never deadlock later?

Answer key with short explanations

1. Response = 5 – 2 = 3; turnaround = 14 – 2 = 12.
2. FIFO. LRU and OPT are stack algorithms and do not show Belady's anomaly.
3. 4 KiB = 4096 = 2^12 bytes, so 12 offset bits.
4. Yes, because Need ≤ Work component-wise; after finishing, add its Allocation back to Work.
5. Only in the classical single-instance-per-resource-type case. With multiple instances, a cycle is not sufficient in general.
6. False. The page table may contain a valid mapping.
7. Better granularity/response in some cases, but more context-switch overhead.
8. Mutual exclusion, hold and wait, no preemption, circular wait.
9. 30/6000 seconds = 0.005 seconds = 5 ms.
10. Indexed allocation.
11. Page number = floor(5000/4096) = 1; offset = 904 bytes.
12. No. It means a safe sequence exists for the current state; future allocations can change the state.

Final revision checklist

- I can calculate CT, TAT, WT and RT from a Gantt chart.
- I can simulate FCFS, SJF/SRTF, Priority and Round Robin.
- I can explain critical-section requirements and trace semaphore operations.
- I can apply Banker's safety algorithm and distinguish safe, unsafe and deadlocked states.
- I can translate logical addresses using page tables and calculate page-table size.
- I can simulate FIFO, LRU and OPT page replacement.
- I can compare file allocation methods and solve disk-head movement questions.

- I have solved official GATE PYQs and reviewed why each incorrect option is wrong.

Important: This guide is a revision resource, not a replacement for the official GATE syllabus or authentic previous-year papers. Always verify syllabus changes for your target exam year and practise exact exam questions separately.